

Part B: PR #482 Review

1. Verbatim Review Comments

● **BLOCKING:** do not merge until these are resolved.

[BLOCKER 1] `cashbackService.ts` **line 14:** `Member.findAll()` **not scoped to** `accountId`

```
const members = await Member.findAll();
```

Loads every member across all brands. This will pay cashback to members of brand B when the cron runs for brand A. `accountId` is validated in the route but never passed into or used by the service.

[BLOCKER 2] `cashbackService.ts` **lines 42-50:** **No idempotency. Running twice pays twice.**

No guard against a member receiving cashback twice for the same week. Retry, operator error, or deploy rollback → every member double-credited. Incident #1 was exactly this pattern.

[BLOCKER 3] **Migration +** `cashbackService.ts` **lines 29, 42-50:** **Float arithmetic on money**

```
netLoss += parseFloat(bet.amount) - parseFloat(bet.payout);  
amount: { type: Sequelize.FLOAT } // migration
```

The PR description states money must go through `bignumber.js` (`src/lib/money.ts` → `dec()`) and is stored as `DECIMAL(36,18)`. This uses `FLOAT` in the schema and `parseFloat` throughout: same class of error as incident #4.

[BLOCKER 4] `cashbackService.ts` **lines 42-50:** **Three writes, no transaction**

```
await wallet.save();           // write 1  
await CashbackPayout.create(); // write 2  
await WalletTx.create();       // write 3
```

Crash between any two writes leaves the ledger inconsistent. All three must be inside a single Sequelize transaction.

[BLOCKER 5] `routes/admin.ts` : **No auth on** `/admin/cashback/run`

No auth middleware visible in this file or in `app.ts`. This endpoint triggers payouts for all members. If externally reachable, anyone can fire it. Confirm existing middleware covers this; if not, add auth before merge.

2. The Conversation

With S.: DM before D. sees the review:

"I left a blocking review on #482: money-critical issues. 15 minutes before D. reads it?"

I open with a question, not an accusation: *"Five blockers: cross-brand leak, no idempotency, float arithmetic. Three match incidents #1 and #4. What were you looking at when you reviewed it?"*

If S. missed it: *"This is exactly why we need a payment PR checklist. You're the right person to write it."* Awkward moment becomes the mandate for something I'm building anyway. If S. pushes back: I listen; update if S. is right. If not: *"I still want these fixed before merge: if this triggers an incident, neither of us will sleep."*

With D.: after S.:

"You followed S.'s advice exactly. The problem isn't your code: payment rules were never written down. System failure, not yours."

Two issues together as a paired discussion, not a lecture. Goal: D. understands *why*, not just *what*.

Systemic fix:

Payment PR checklist in the repo as a GitHub PR template, auto-triggered for `**/wallet/**`, `**/payments/**`, `**/cashback*`. Six items: accountId scoping, idempotency guard, DECIMAL/bignumber, transaction boundary, structured error logging, admin auth. S. authors it: if S. writes it, S. enforces it, and D. self-catches these issues before the next PR is created.